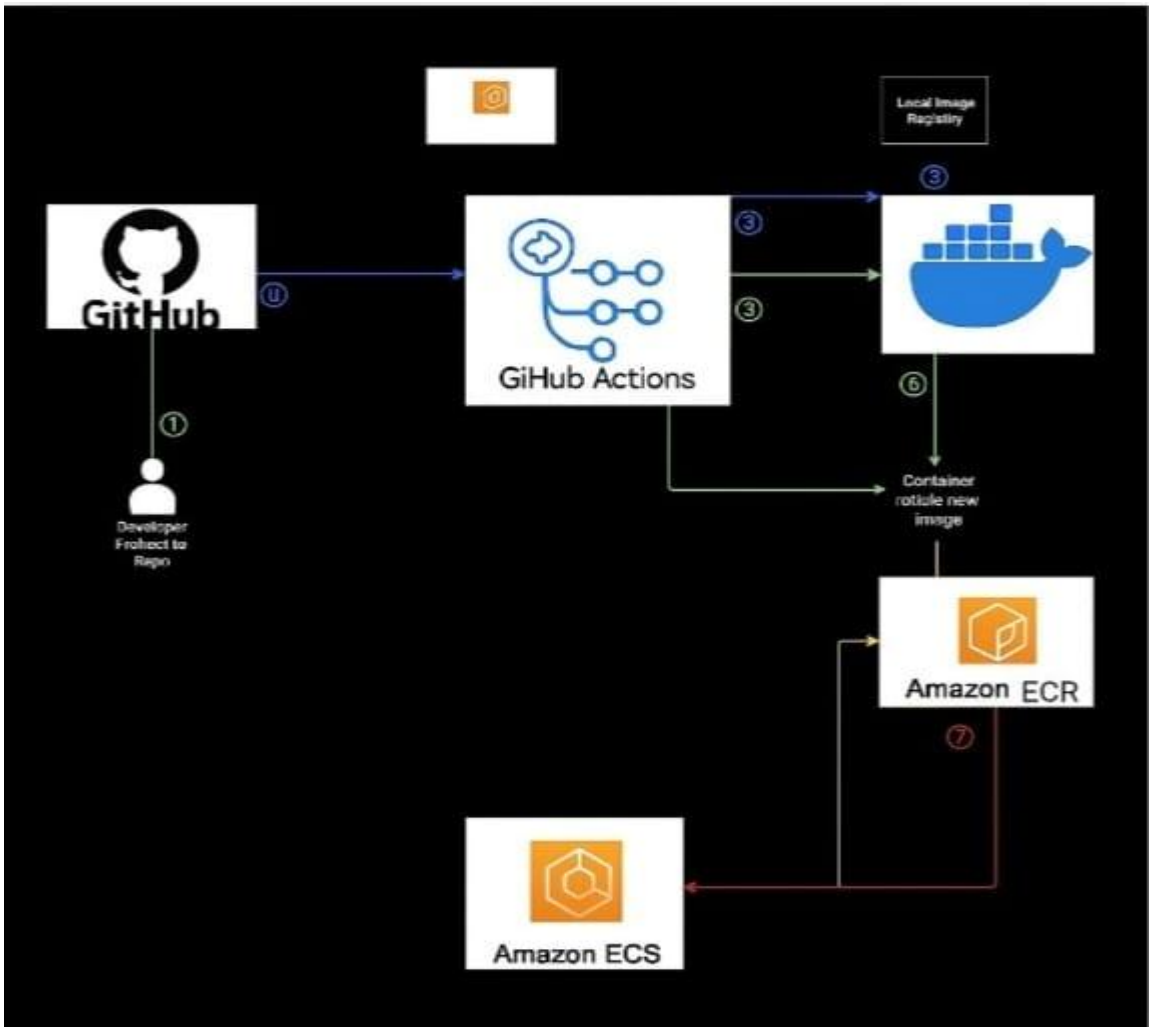
PROJECT – 6

Automated CI/CD Deployment of Java Application Using GitHub Actions, Docker, Amazon ECR, and Amazon ECS

Presentation By : M.Karthik Reddy

Batch 141

Automated CI/CD Deployment of Java Application Using GitHub Actions, Docker, Amazon ECR, and Amazon ECS



Tools Used:

- **GitHub** - Source code manage
- **GitHub Actions** - CI/CD automation
- **Docker** - Containerization of application
- **Amazon ECR** - Docker image registry
- **Amazon ECS** - Serverless container deployment
- **IAM** - Authentication and authorization
- **CloudWatch** - Logging and monitoring

INTRODUCTION

Modern software development requires fast, reliable, and automated deployment processes to meet business demands. Manual application deployment is time-consuming, error-prone, and difficult to scale. To overcome these challenges, Continuous Integration and Continuous Deployment (CI/CD) pipelines are widely adopted in DevOps practices.

This project focuses on designing and implementing an automated CI/CD pipeline for a Train Ticket Application using containerization and cloud-native services. The main objective is to automate the complete software delivery lifecycle starting from code commit to production deployment without manual intervention.

The application source code is stored in a GitHub repository. Whenever a developer pushes new code, GitHub Actions automatically triggers the pipeline. The pipeline builds a Docker image using the application source code and Dockerfile, ensuring consistency across environments. The built image is then pushed to Amazon Elastic Container Registry (ECR), which acts as a secure private image repository.

For deployment, Amazon Elastic Container Service (ECS) with Fargate launch type is used. Fargate enables serverless container execution where AWS manages the underlying infrastructure, eliminating the need for EC2 instance management. ECS pulls the latest image from ECR and deploys the container as a running task. This approach improves scalability, reliability, and operational efficiency.

The implemented CI/CD pipeline provides several benefits including faster release cycles, reduced deployment errors, better version control, and improved application availability. By combining GitHub Actions, Docker, Amazon ECR, and Amazon ECS Fargate, this project demonstrates a real-world DevOps workflow that aligns with industry standards and best practices.

Overall, this project highlights the importance of automation, containerization, and cloud-native deployment strategies in modern software engineering environments and serves as a practical implementation of end-to-end CI/CD deployment.

Deployment Architecture Flow:

When Developer Push Code → GitHub Repository
→ GitHub Actions Pipeline (CI/CD)
→ Docker Image Build
→ Push to Amazon ECR
→ ECS Deployment
→ Running Application Container

PROCESS OF DEPLOY WAR FILE IN TOMCAT USING GITHUB ACTIONS

STEP – 1: CREATE AMAZON ECR

- Created Private Docker image repository, Label the name as “project-image-repo”
- Used to store application container images.

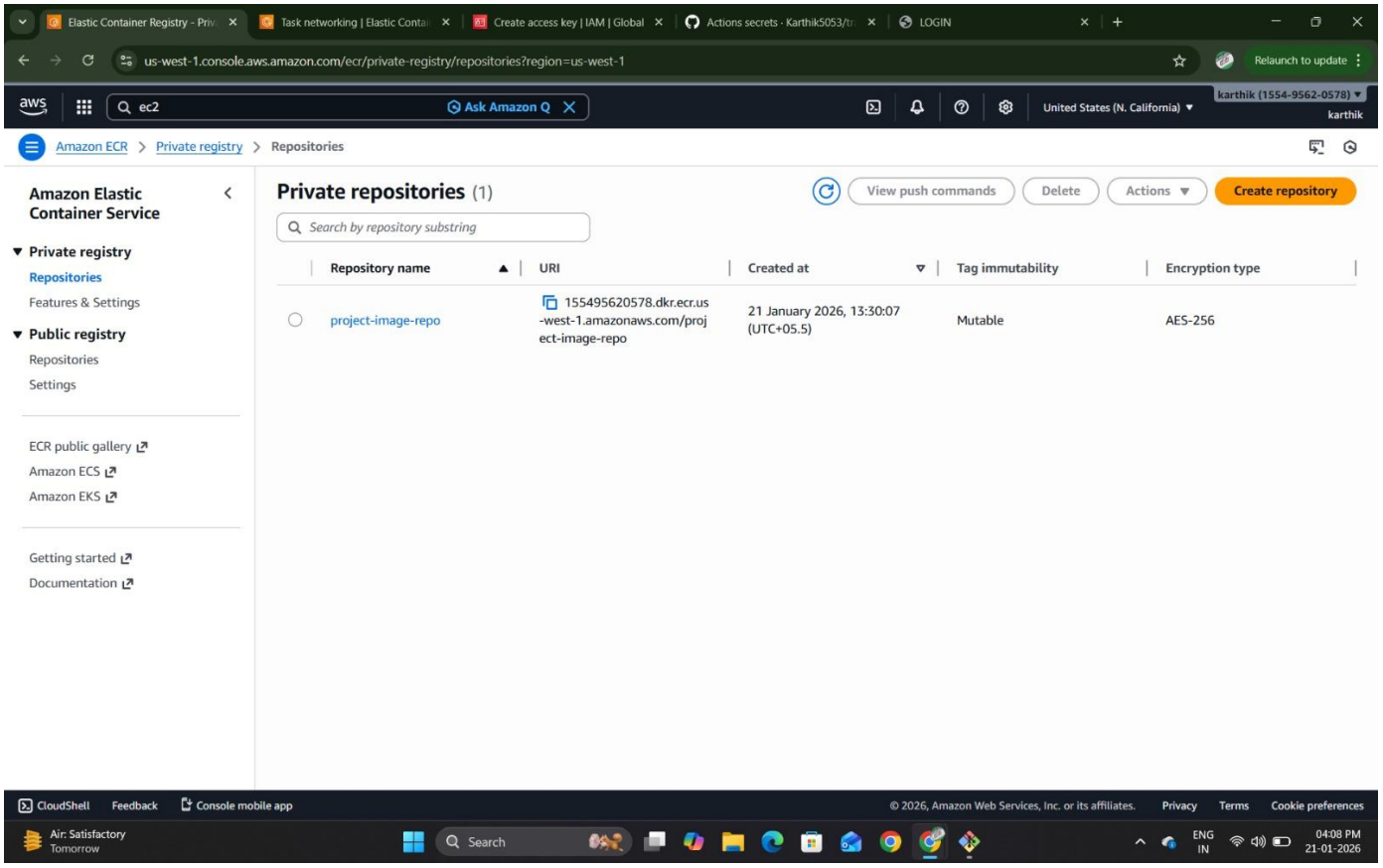


Fig-1: ECR Repo

STEP – 2: CREATE AMAZON ECS CLUSTER

- Created ECS cluster using Fargate launch type.
- Label the name as cluster as “projec-cluster”.

STEP – 3: CREATE TASK DEFINITION

- Configured container image.
- CPU and memory allocation.
- Port mapping.

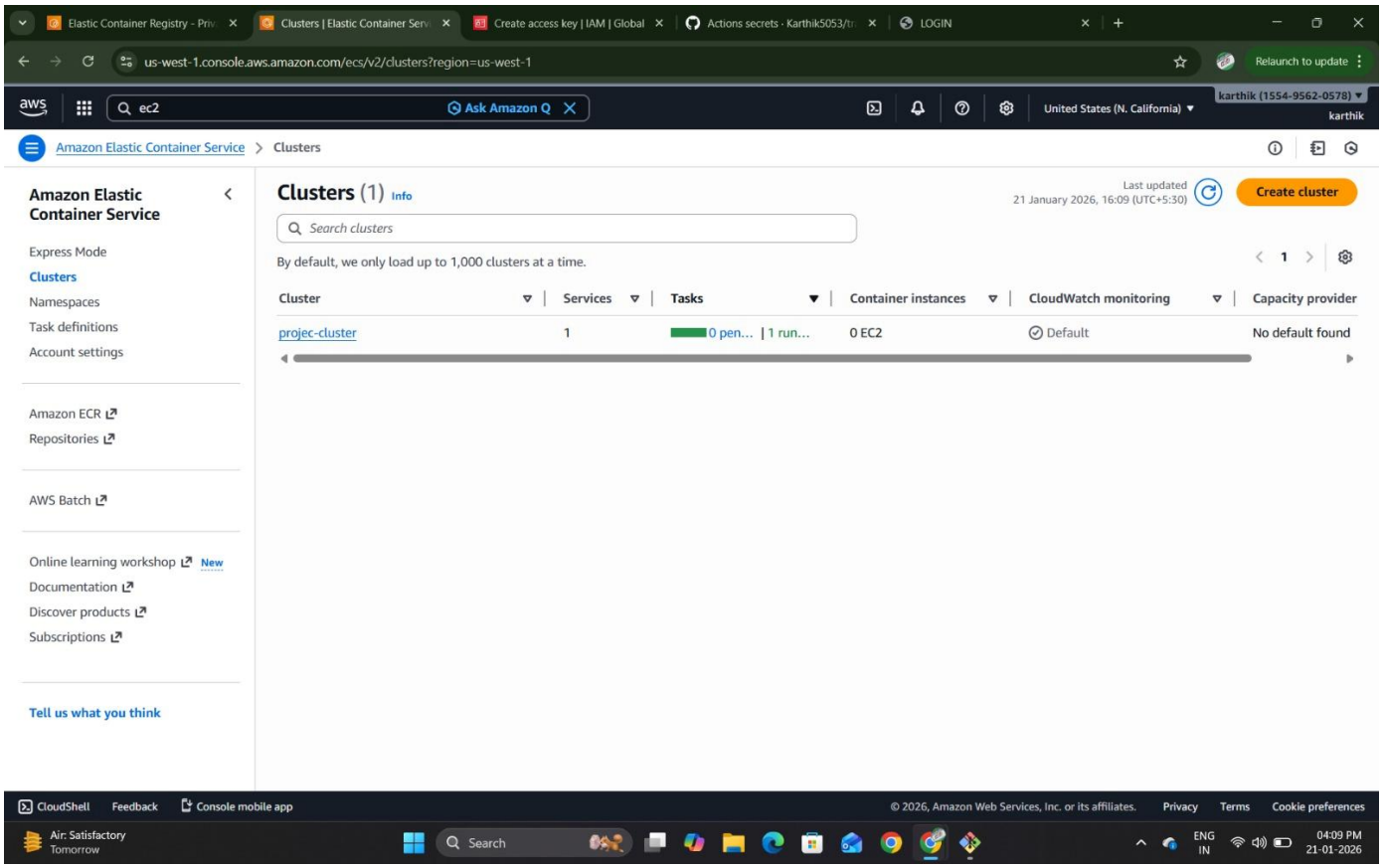


Fig-2: Create Cluster

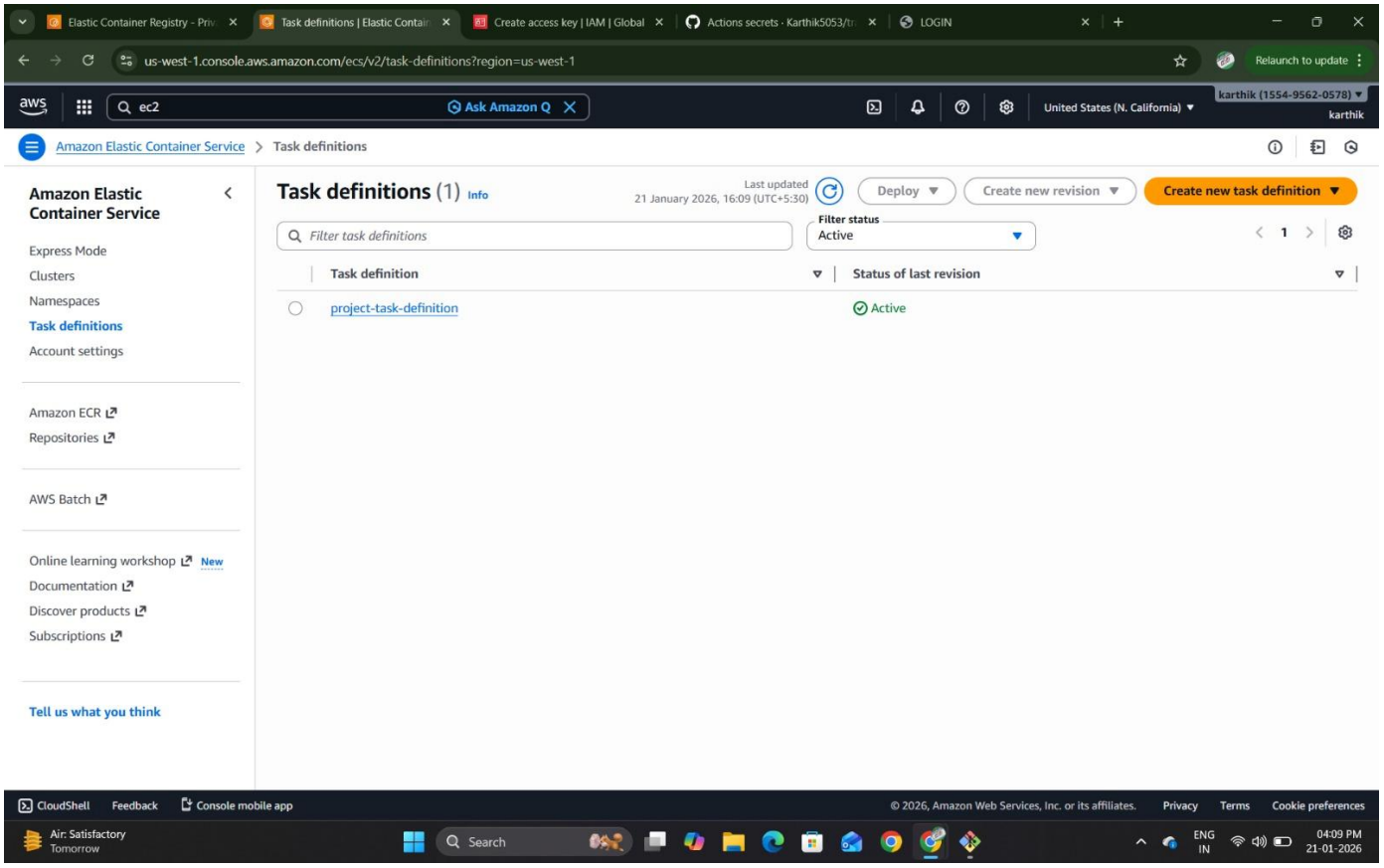


Fig-3: Create Task Definition

STEP – 4: CREATE ECS SERVICE

- Created service to maintain running container.
- Label name as “project-task-definition-service”.

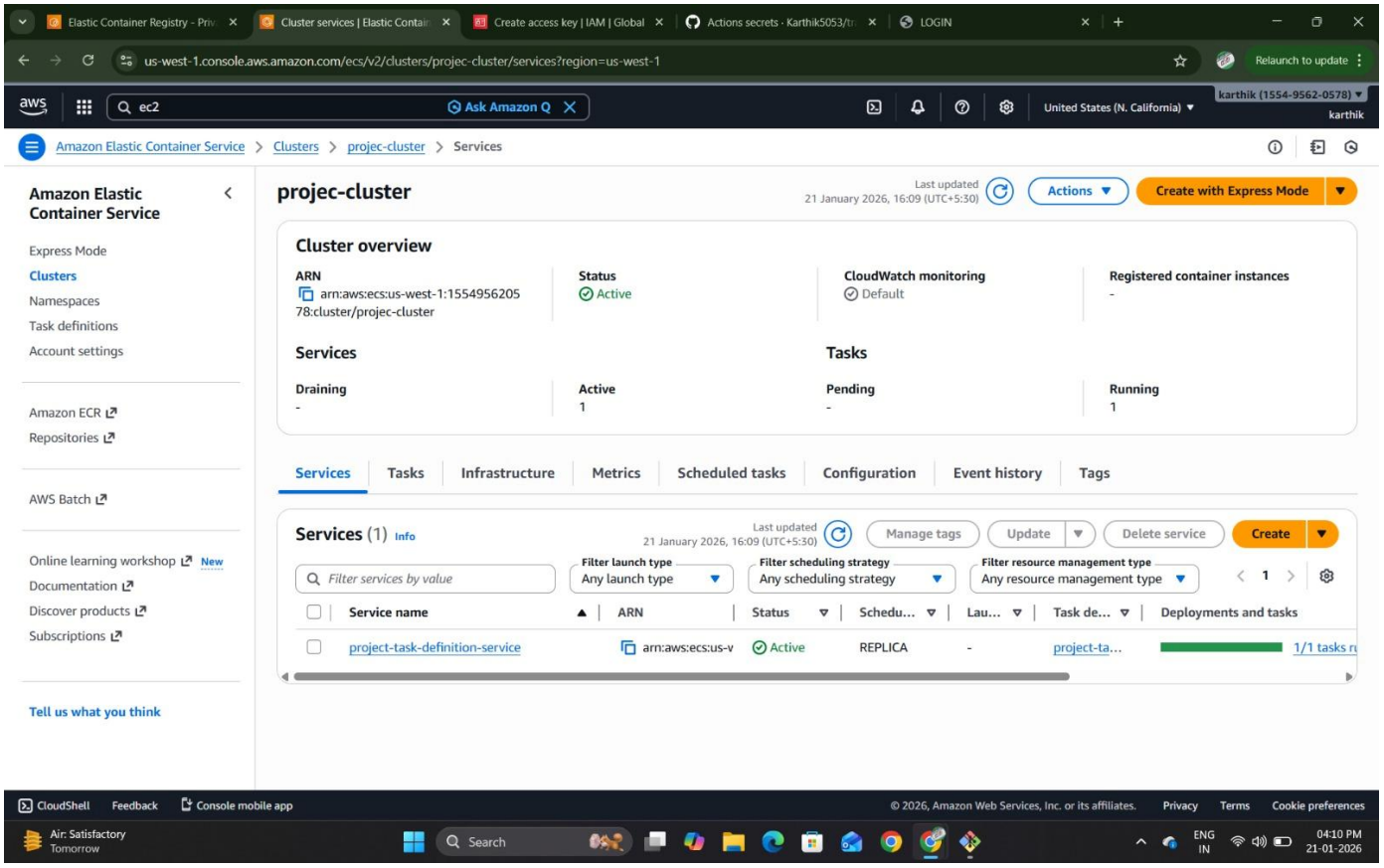


Fig-4: Create ECS Service

STEP – 5: CREATE IAM CREDENTIALS

- It can obtain AWS Access Key and AWS Secret Key.

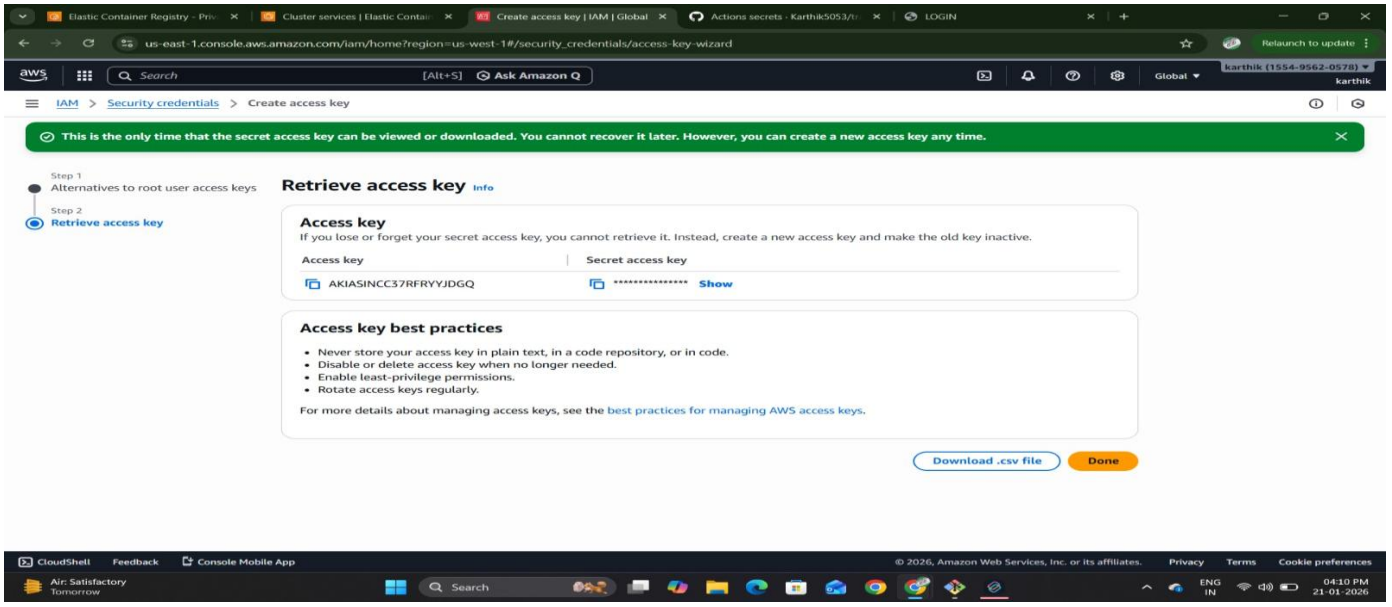


Fig-5: IAM Credentials

STEP – 6: CONFIGURE GITHUB REPOSITORY

- Source code in a GitHub repository.
- Add Dockerfile Inside the repo
- Create .github/workflows/main.yaml file was created to define the CI/CD pipeline.
- Add Secrets actions repository for CI/CD pipeline.

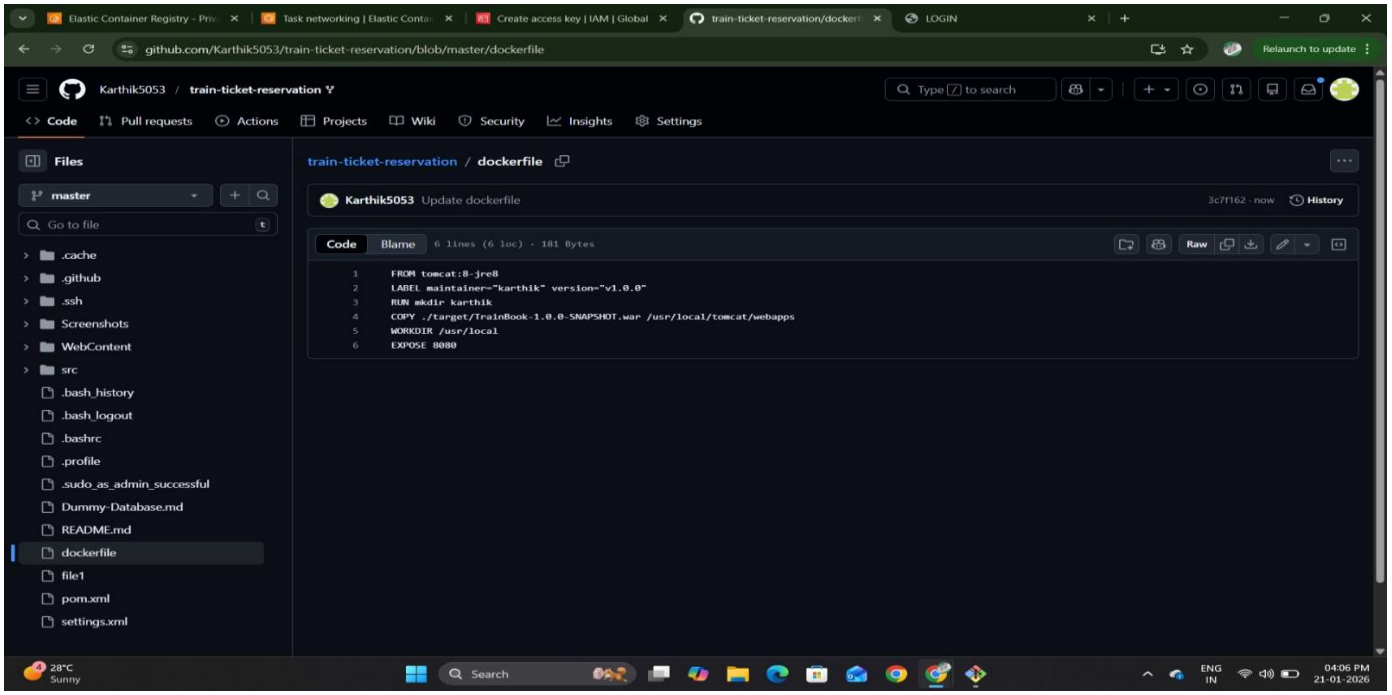


Fig-6: Create Docker File Inside a Repo

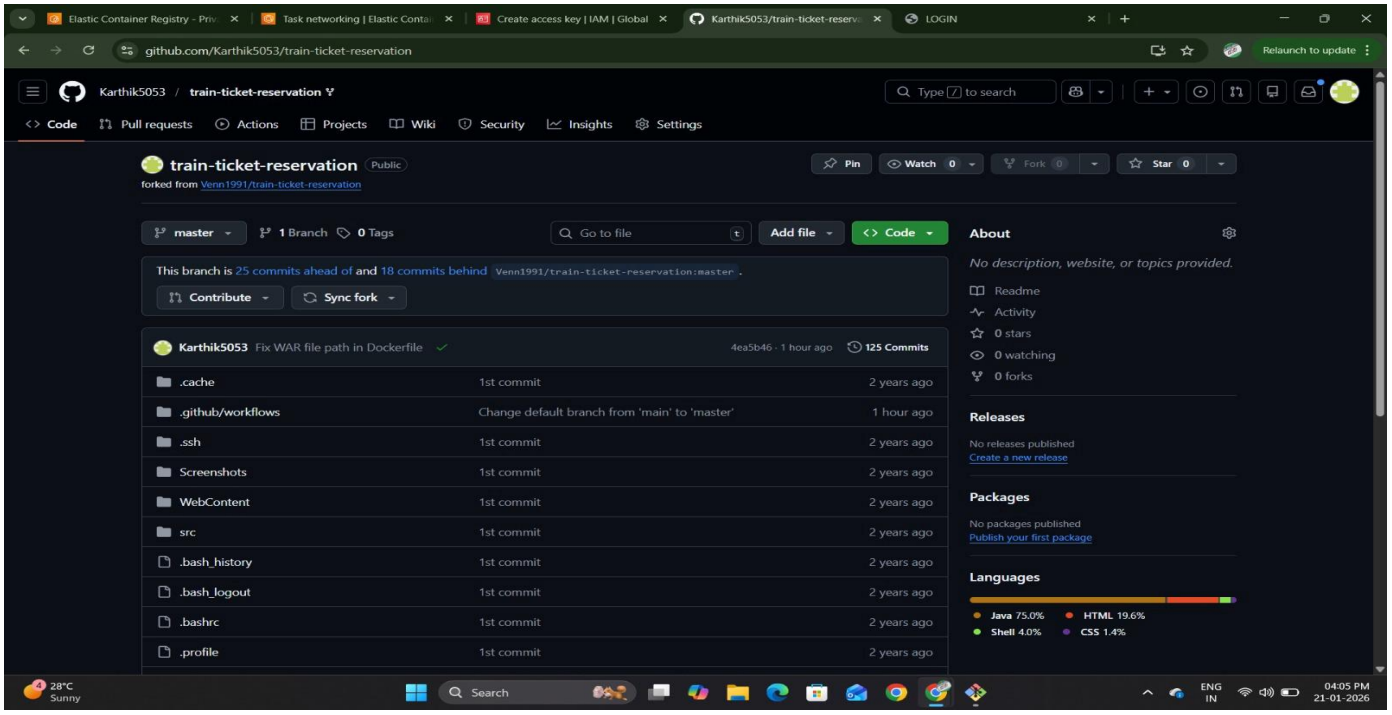


Fig-7: Source Code

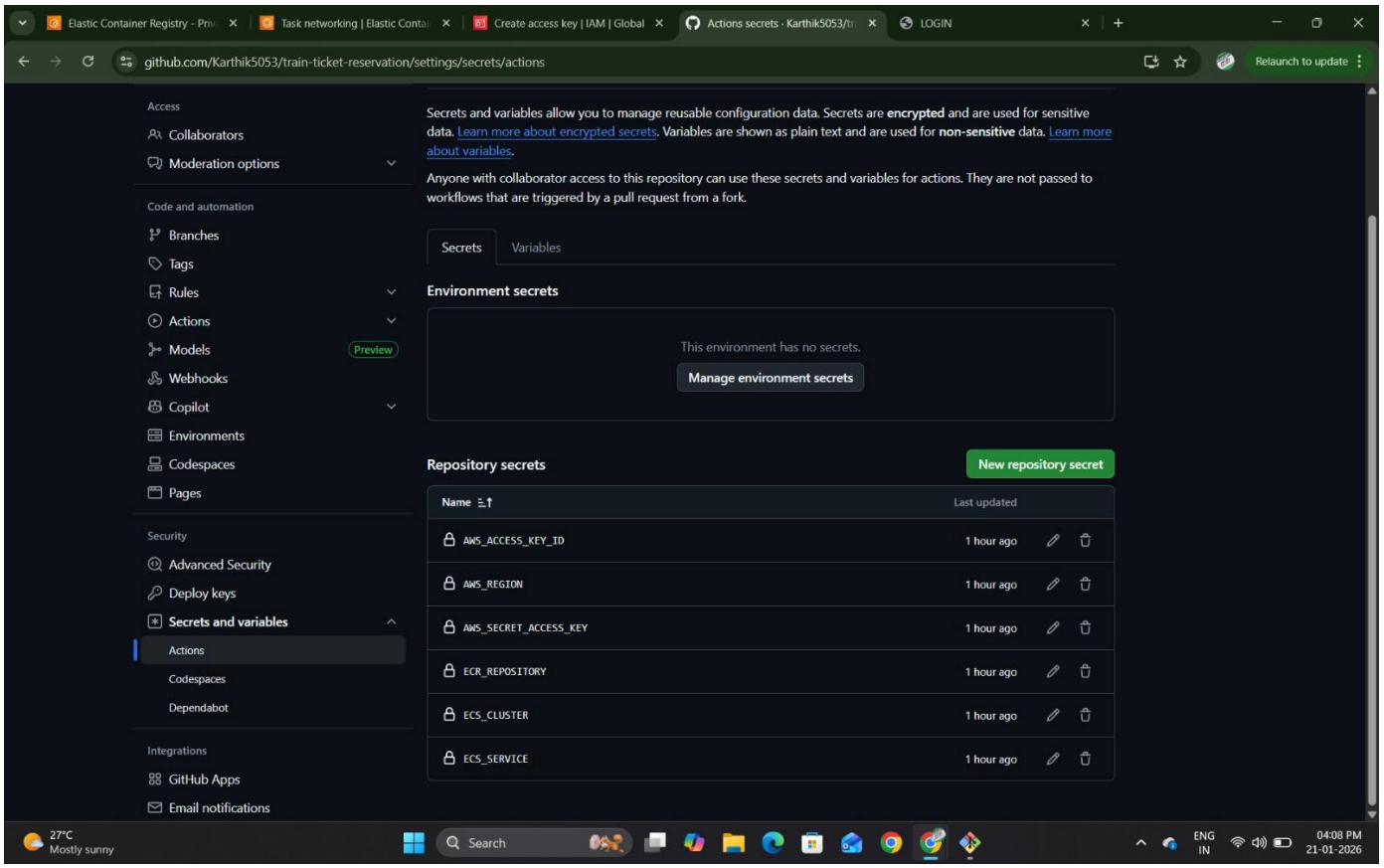


Fig-8: Add Secret Actions

STEP – 7: CI/CD PIPELINE USING GITHUB ACTIONS

- The pipeline checks out the code.
- Docker Image Build.
- Authentication to AWS ECR.
- Push Image to Amazon ECR.
- ECS Deployment Trigger.
- Container Execution on Fargate.

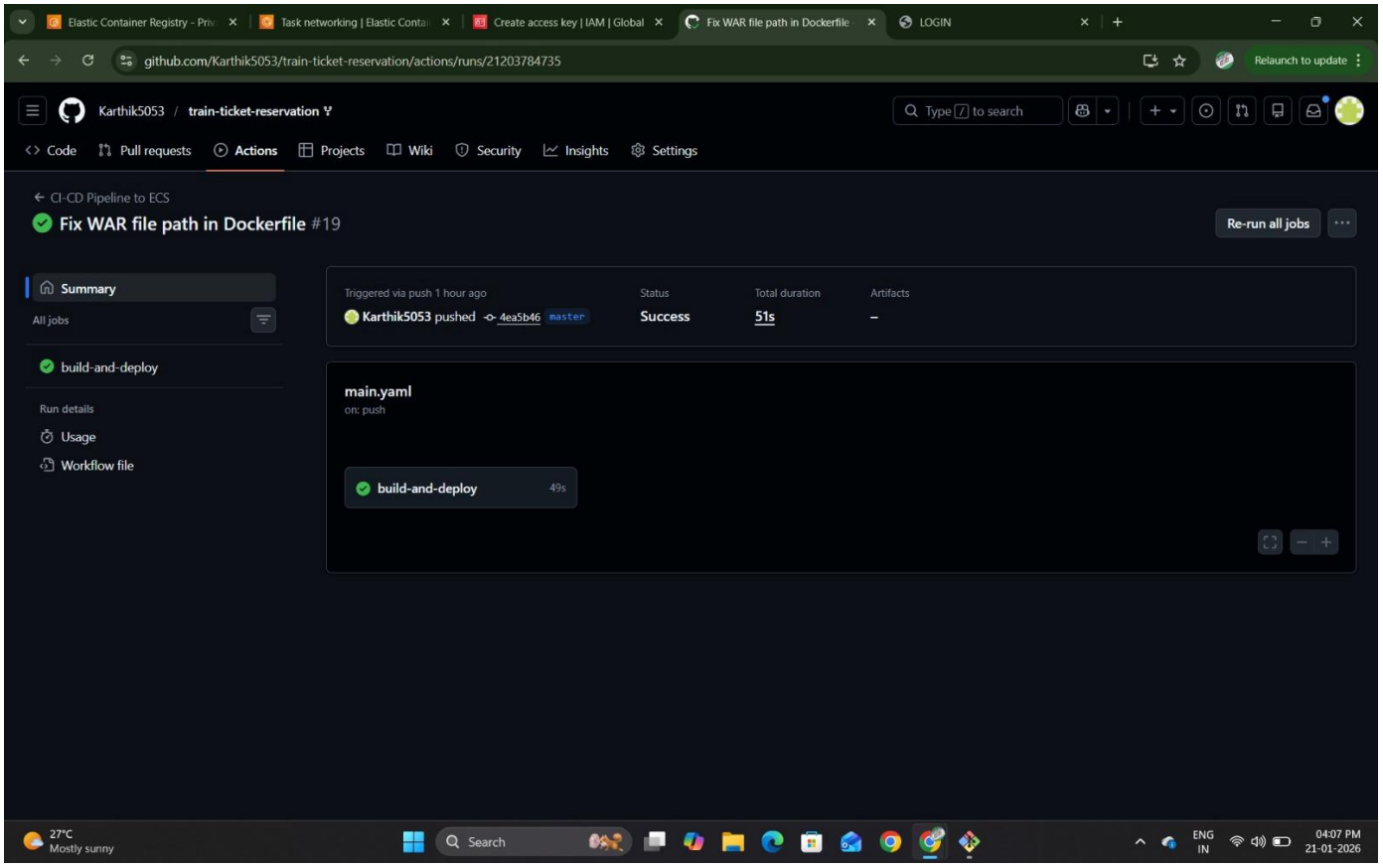


Fig-9: Bulid Github Actions

STEP – 8: DEPLOYMENT VERIFICATION PROCESS (After pipeline completion):

A) ECS Validate

- Verified ECS service status as Active.
- Confirmed running task status.

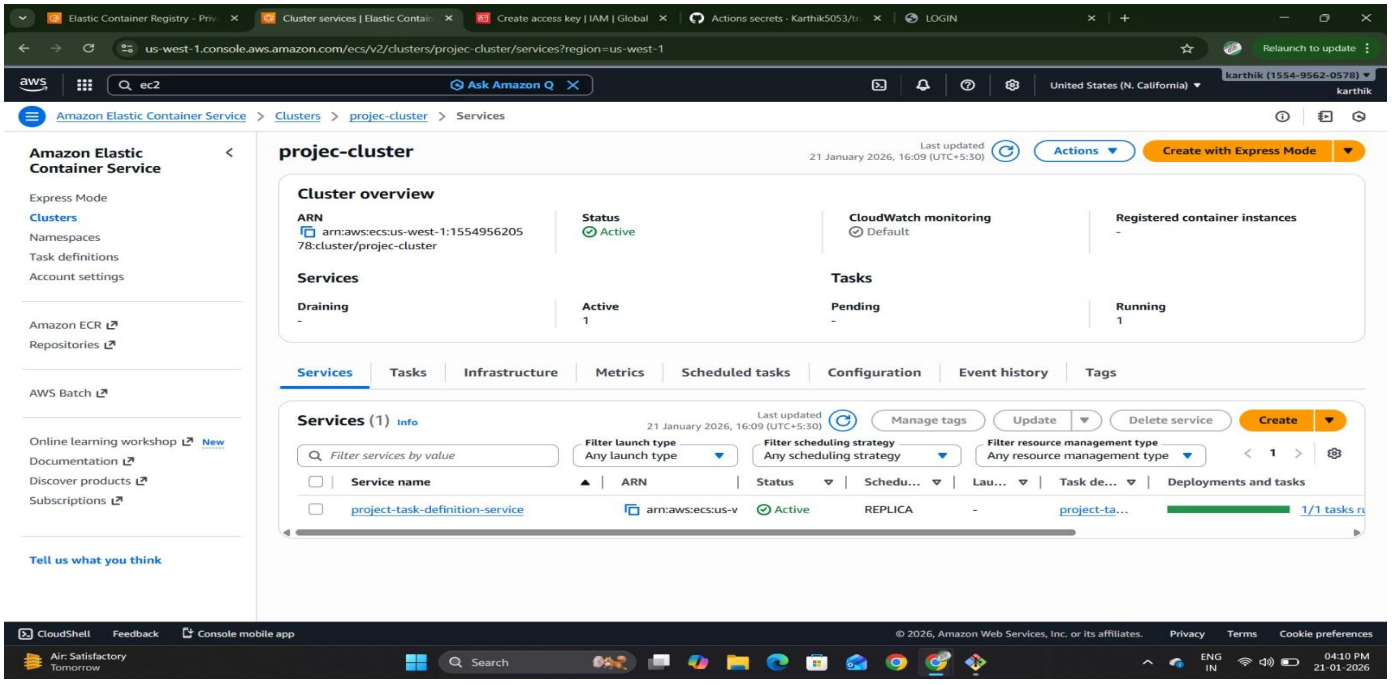


Fig-10: Verified ECS service status as Active

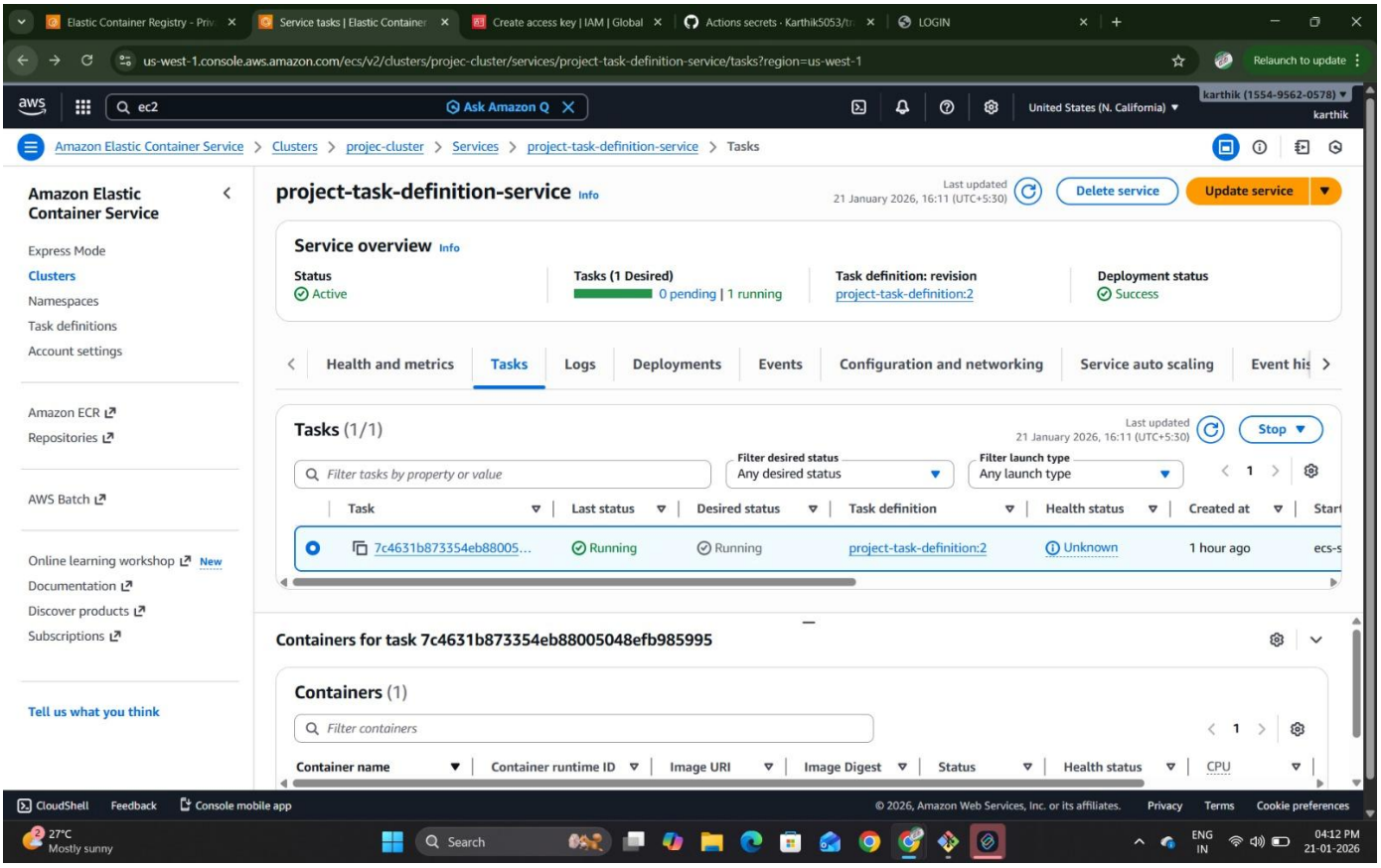


Fig-11: Running Task Status

B) Application Testing

- Retrieved public IP from ECS task.
- Accessed application through browser.

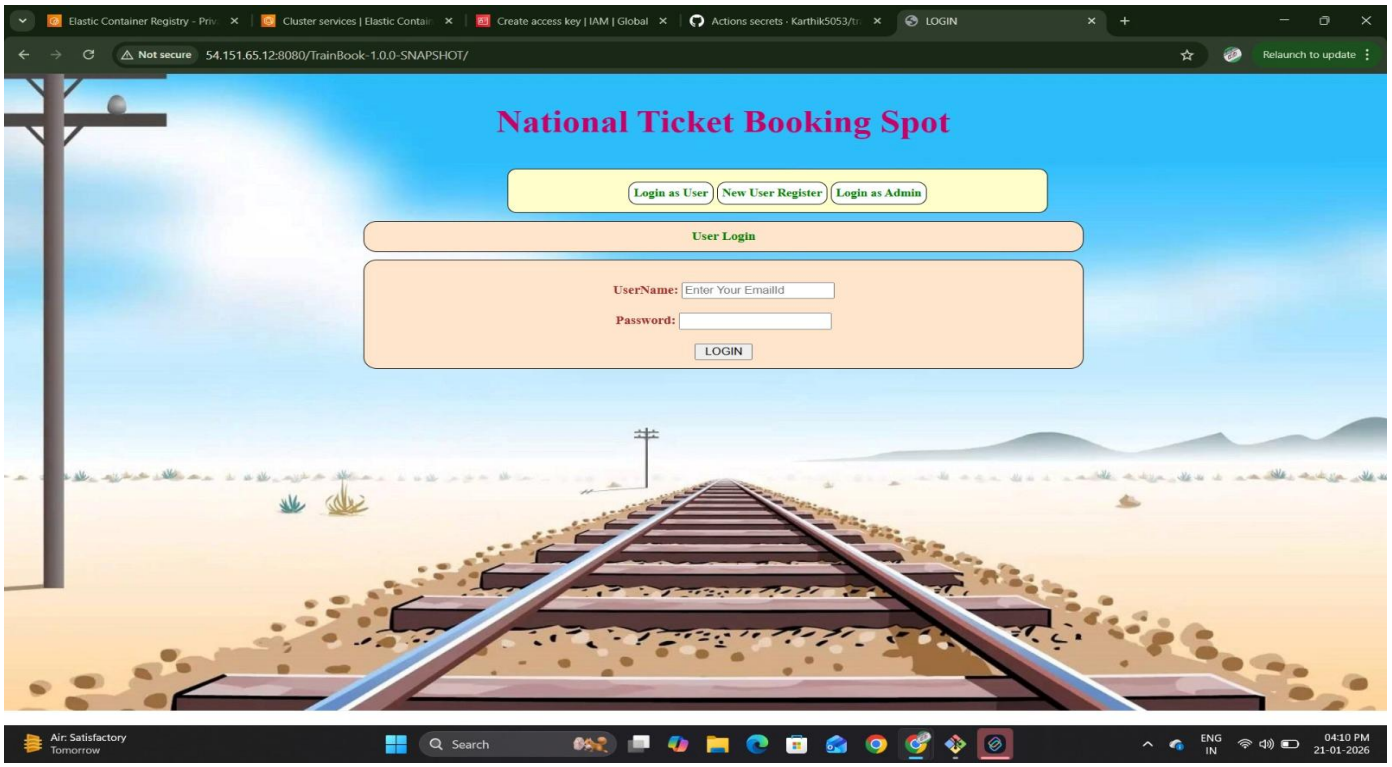


Fig-12: Accessed application through browser

C) Image Verification

- Confirmed updated image version running in ECS task details.

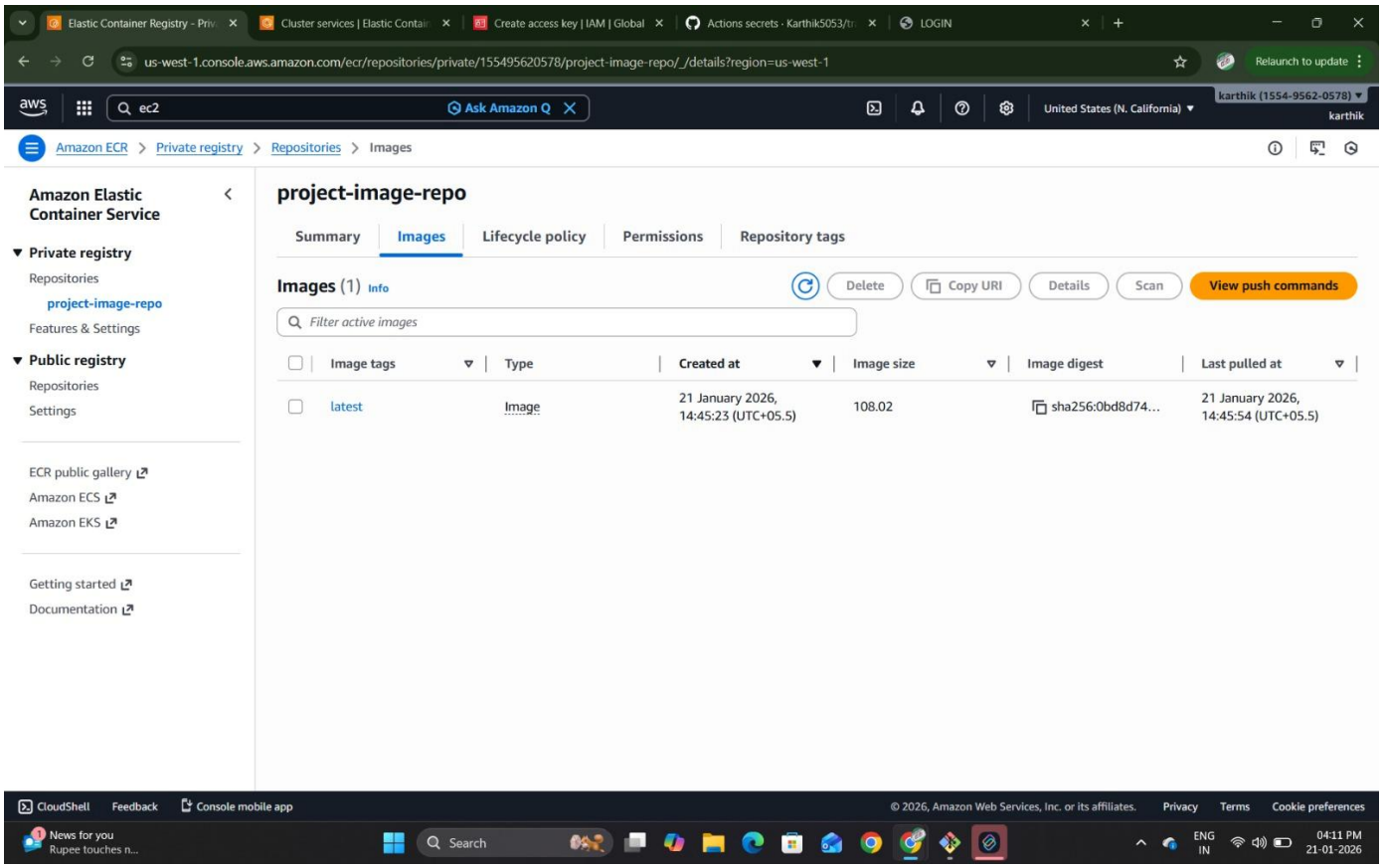


Fig-13: Confirmed updated image In ECS

CONCLUSION

This project successfully demonstrates the implementation of a fully automated CI/CD pipeline for deploying a Train Ticket Application using modern DevOps tools and cloud services. By integrating GitHub Actions, Docker, Amazon ECR, and Amazon ECS Fargate, the deployment process was transformed from a manual and error-prone workflow into an efficient, reliable, and automated system.

The use of containerization ensured application consistency across different environments, eliminating dependency-related issues. GitHub Actions enabled continuous integration by automatically building and validating application code whenever changes were pushed to the repository. Amazon ECR provided a secure and scalable image storage solution, while Amazon ECS Fargate enabled serverless container execution, allowing the application to run without the need to manage physical servers or virtual machines.

This automated deployment pipeline significantly improved deployment speed, reduced operational overhead, and minimized human errors. It also enabled faster feedback cycles and easier rollback mechanisms by maintaining versioned container images. The project reflects real-world DevOps practices used in enterprise environments, emphasizing the importance of automation, scalability, and reliability in software delivery.

Overall, this implementation proves that adopting CI/CD pipelines and cloud-native technologies is essential for building modern, production-ready applications. The project serves as a strong foundation for further enhancements such as load balancer integration, auto-scaling, blue-green deployments, and advanced monitoring. It also provides valuable hands-on experience in DevOps workflows and cloud infrastructure management, preparing the system for real-world production use.